Session 02

Web Programming

Introduction to Frontend Development

Instructors

Elie ABI HANNA DAHER
Tech Lead @ Societe Generale
eliedhr@gmail.com
github.com/elieahd

Lucas Bilal EL CHAMI
Tech Lead @ Uprizon
bilalchami1@gmail.com
github.com/bilal-elchami

SOCIETE GENERALE

Uprizon
A bold company

01

Introduction to Frontend Development

What is Frontend Development?

- Designing and implementing the **user interface** and **user experience** of a website or web application.
- Most used languages are **HTML**, **CSS**, and **JavaScript** to create visually appealing and interactive elements.
- Frontend developers focus on ensuring that websites are **responsive**, **accessible**, and **user-friendly** across **different devices and platforms**.

Importance of Frontend Dev in Web Apps

1. **User Experience (UX):** It shapes how users interact with a website or web app, enhancing satisfaction and encouraging return visits.
2. **Accessibility:** It ensures websites are accessible to users with disabilities, adhering to coding best practices like semantic HTML and providing alternative text for images.
3. **Responsiveness:** Frontend developers implement responsive design techniques to ensure seamless adaptation to various devices and screen sizes.
4. **Performance:** Optimizing frontend code and assets improves loading times and overall performance.

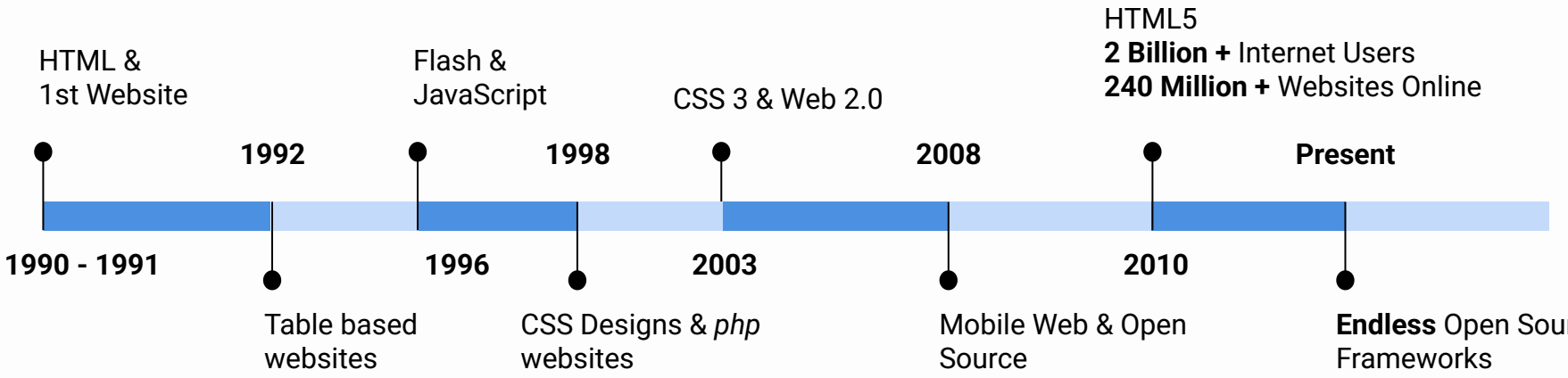
Key Skills for Frontend Developers

- Proficiency in **HTML**, **CSS**, **JavaScript** and **TypeScript**
- Knowledge of CSS Frameworks
- Understanding of **Responsive Design**
- Experience with Frontend Frameworks/Libraries
- **Version Control!!** 🚨
- Basic **Design** Skills
- **Debugging** and Problem-Solving
- API Integration
- Continuous Learning

02

Overview of Frontend Technologies

Quick History



How to choose your Frontend Dev Stack?

In a world full of frontend stacks, choose the stack that won't be against you!

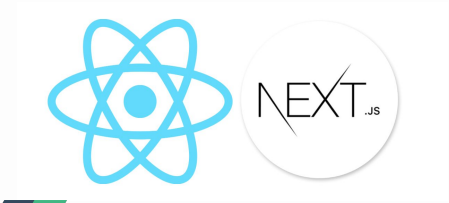
Why Angular & Bootstrap for this course?

Angular offers:

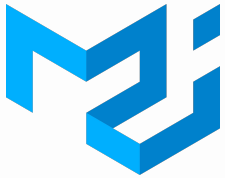
1. **Robustness** backed by Google.
2. Performance optimizations.
3. **Modular architecture** for **maintainability**.
4. TypeScript support.
5. Large community and resources.
6. **Enterprise-ready** features.
7. Cross-platform development capabilities.

Other options would be

JavaScript / TypeScript Frameworks:



CSS Frameworks:



03

Introduction to Angular

What is Angular?

Angular is a development platform, built on **TypeScript**.

As a platform, Angular includes:

- A **component-based framework** for building scalable web applications
- A collection of well-integrated libraries that cover a wide variety of features, including **routing**, **forms management**, **client-server communication**, and more
- A suite of developer tools to help you develop, build, test, and update your code

With Angular, you can start with a **single-developer project** and scale it easily to an **enterprise-level** application.

AngularJS

- Released in 2010 by Google.
- A **JavaScript** -based open-source front-end web framework.
- Uses **MVC (Model View Controller) architecture**.
- Known for its two-way data binding.
- Syntax-heavy, relies heavily on ng directives.

Angular

- Released in 2016 by the Angular Team at Google.
- A complete rewrite of AngularJS in **TypeScript**.
- Follows a **component-based architecture**.
- Emphasizes modularity and improved performance.

04

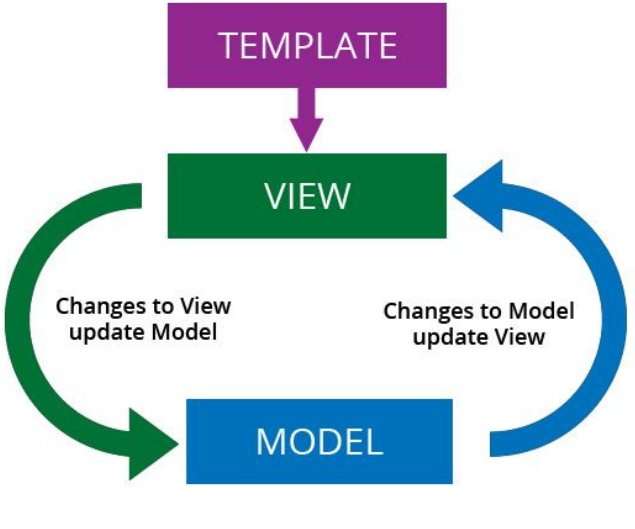
Key Features of Angular

Two-Way Data Binding

One of Angular's most prominent features is its **two-way data binding**.

Two-way data binding allows automatic synchronization of data between the **model** and the **view**, ensuring that any changes in the model are immediately reflected in the view and vice versa.

This simplifies development and reduces the need for **manual DOM manipulation**.



Properties Binding Syntax

Angular provides three categories of data binding according to the direction of data flow:

- From source to view
- From view to source
- In a two-way sequence of view to source to view

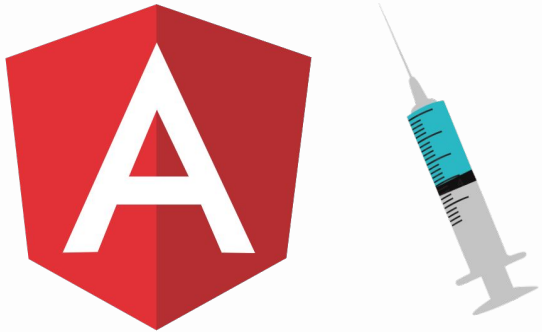
TYPE	SYNTAX	CATEGORY
Interpolation Property Attribute Class Style	`{{expression}}` `[target]="expression"`	One-way from data source to view target
Event	`(target)="statement"`	One-way from view target to data source
Two-way	`[(target)]="expression"`	Two-way

Dependency Injection

Angular utilizes a robust **Dependency Injection** system, which is a software **design pattern** that promotes loose coupling and modular development.

Dependency injection allows components and services to be **injected with their dependencies** rather than creating them internally.

This promotes code reusability, testability, and maintainability by enabling **easier management of dependencies** and facilitating easier unit testing.

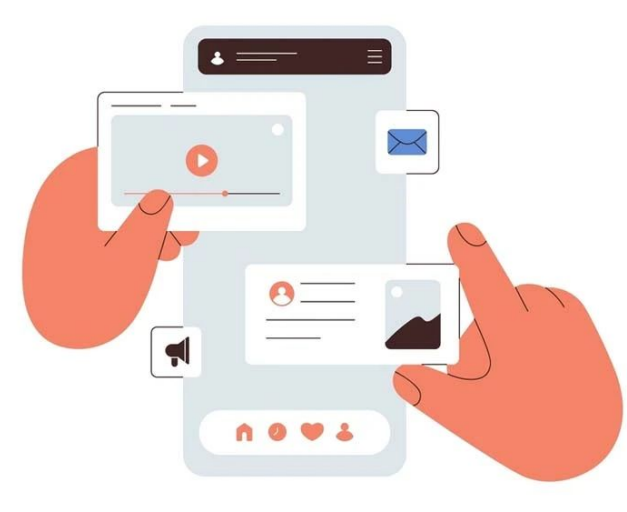


Component-Based Architecture

Angular follows a **component-based architecture** where the application is built using **reusable** and **encapsulated components**.

Components encapsulate **the template, logic, and styling** of a part of the UI, making it easier to manage and reuse code.

This modular approach fosters better organization of code, enhances **maintainability**, and **promotes collaboration** among developers.



Component-Based Architecture

Angular components are made out of 3 main files.

- TypeScript File.
- HTML Template File.
- CSS Styling File.

A component is defined by the `@Component` annotation. This annotation takes as parameters the component **selector** (html tag), the **template URL** or direct **inline HTML** and the **styles URL** or direct **inline CSS**.

```
1   import { Component } from '@angular/core';
2
    You, 52 seconds ago | 1 author (You)
3   @Component({
4     selector: 'app-root',
5     templateUrl: './app.component.html',
6     styleUrl: './app.component.css'
7   })
8   export class AppComponent {
9     greeting = 'Hi mom 👋🏽';
10  }
```

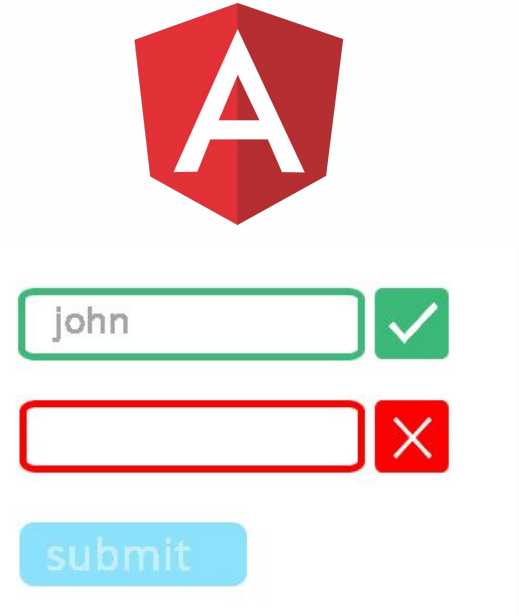
```
<main class="main">
  <h1 *ngIf="greeting">{{ greeting }}</h1>
</main>
```

Reactive Forms

Angular offers robust support for building reactive forms, allowing developers to create **dynamic and interactive forms** with ease.

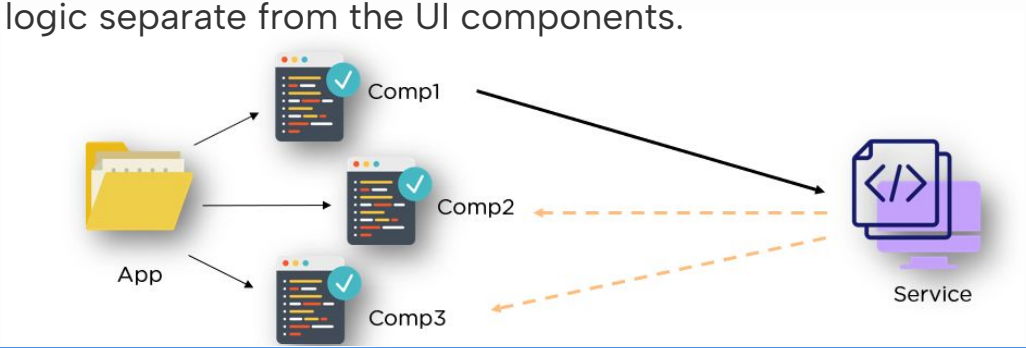
Reactive forms use a reactive programming approach, where form inputs are treated as streams of data that can be **manipulated and validated asynchronously**.

This enables developers to build complex forms with **real-time validation**, **error handling**, and **data synchronization**.



Services

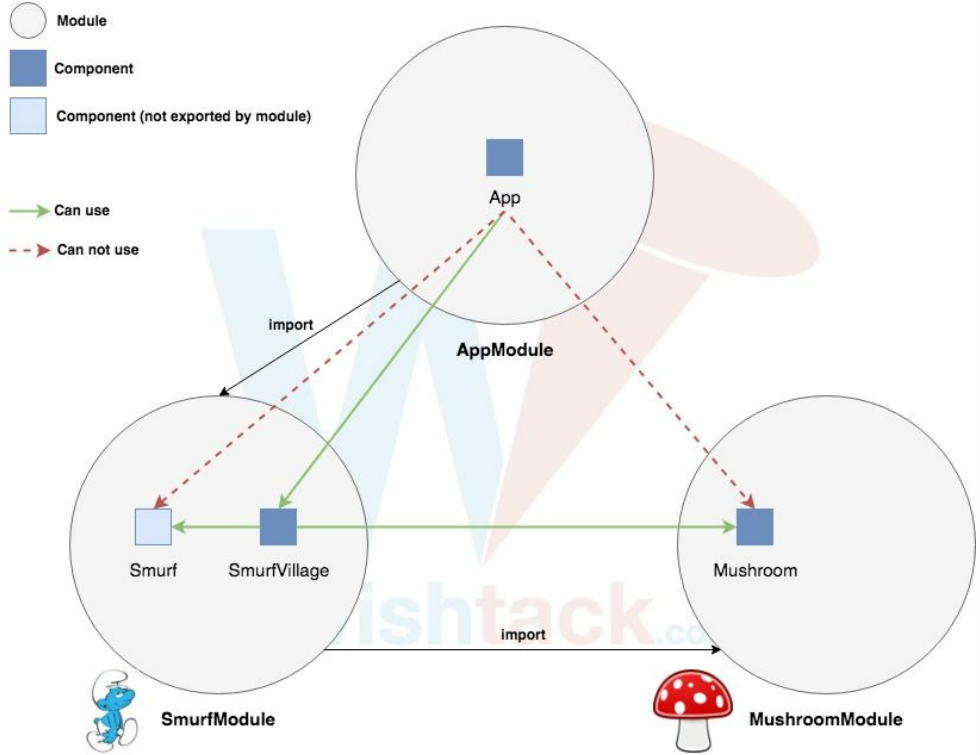
- Services in Angular are **reusable, injectable objects** that provide shared functionality across components.
- Services encapsulate **business logic**, **data access operations**, and other common functionalities.
- Services promote **code reuse**, **maintainability**, and **separation of concerns** by keeping the application's logic separate from the UI components.



Modules

- An Angular application includes at least one code module, typically the `AppModule`.
- Angular itself is made of several modules such as `Core`, `FormsModule`, `HttpClientModule`, `RouterModule`, and more.
- The libraries available for Angular are generally structured as modules.
- Breaking down an application into modules enables it to be loaded progressively as the user navigates through and interacts with different parts of the application.

Modules


Module
Component
Component (not exported by module)
Can use
Can not use
App
import
AppModule
Smurf
SmurfVillage
Mushroom
import
SmurfModule
MushroomModule

Modules

The main module in Angular is the `AppModule`

The main attributes of a module:

- `declarations` → Used to declare **components**.
- `imports` → Used to import other **modules**.
- `exports` → Used to export **components**.
- `providers` → Used to define the **services**.
- `bootstrap` → The **Main Component** which the application will launch at startup.

```
import { NgModule }      from
'@angular/core';
import { BrowserModule } from
'@angular/platform-browser';

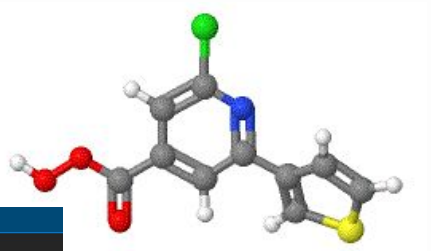
@NgModule({
 imports:      [ BrowserModule ],
 providers:    [ Logger ],
 declarations: [ AppComponent ],
 exports:      [ AppComponent ],
 bootstrap:    [ AppComponent ]
})

export class AppModule { }
```

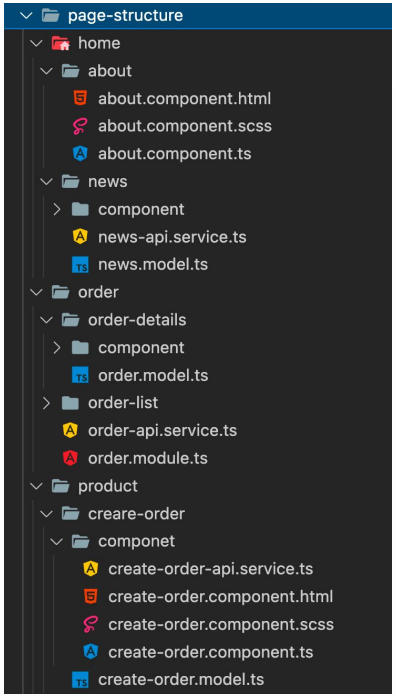
05

Setting Up the Development Environment

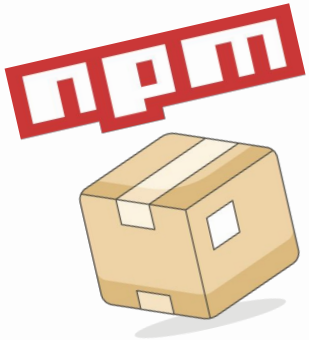
Anatomy of an Angular Project



The anatomy of an Angular project typically consists of several key directories and files, each serving a specific purpose in the development, building, and deployment process. Here's a breakdown of the typical structure of an Angular project:

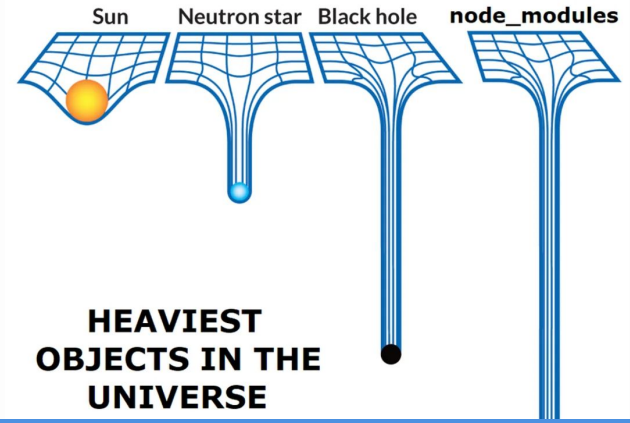


Node Modules

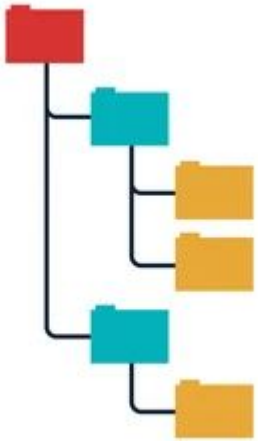


`node_modules`: This directory contains all the npm packages installed as dependencies for the project.

It's usually generated automatically when you run **`npm install`** or **`yarn install`** to install dependencies.



- **`src`**: This directory is where most of the development work occurs. It contains the source code for your Angular application. Inside the src directory, you'll find:
    - **`app`**: This directory contains the components, modules, services, and other code that make up your application. It usually includes:
        - **`components`**: Individual components.
        - **`modules`**: Angular modules that organize your application into logical units.
        - **`services`**: Angular services that provide functionality used across multiple components.
        - **`views`**: Additional views or pages for your application.
        - **`assets`**: Static assets such as images, fonts, or CSS files used by your application.
        - **`app-routing.module.ts`**: The Angular router module that defines the routes for your application.
        - **`app.ts`**: The root component of your application.
        - **`app.module.ts`**: The main Angular module that bootstraps your application.



- **`angular.json`**: This file is the Angular CLI configuration file that defines settings and options for building and serving your Angular application.
- **`package.json`**: This file contains metadata about the project and its dependencies. It also includes scripts for running tasks.
- **`tsconfig.json`**: This file specifies the TypeScript compiler options for the project.
- **`tslint.json`**: This file contains configurations for the TSLint which is an analysis tool that checks TypeScript code for potential errors or code style violations.

Getting Started

Prerequisites:

To run your project locally, you need the following installed on your computer:

- Node.js and npm.
- TypeScript `npm install -g typescript`.
- The Angular CLI `npm install -g @angular/cli`.
- Visual Studio Code.
- Angular Language Service VSCode Extension.
- Git

See more: https://angular.io/guide/setup-local

06

Creating Your First Angular App

Creating an Angular Boilerplate App

Run ng new followed by the application name:

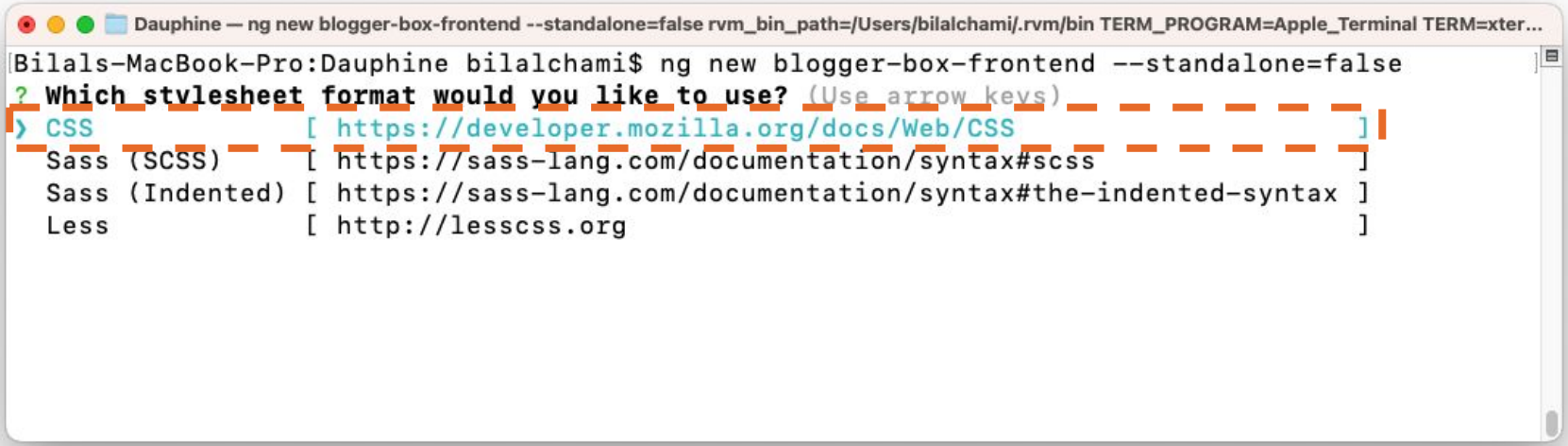
```
ng new blogger-box-frontend --standalone=false
```.

ng new prompts you for information about features to include in the initial project.

ng new also creates the following workspace and starter project files:

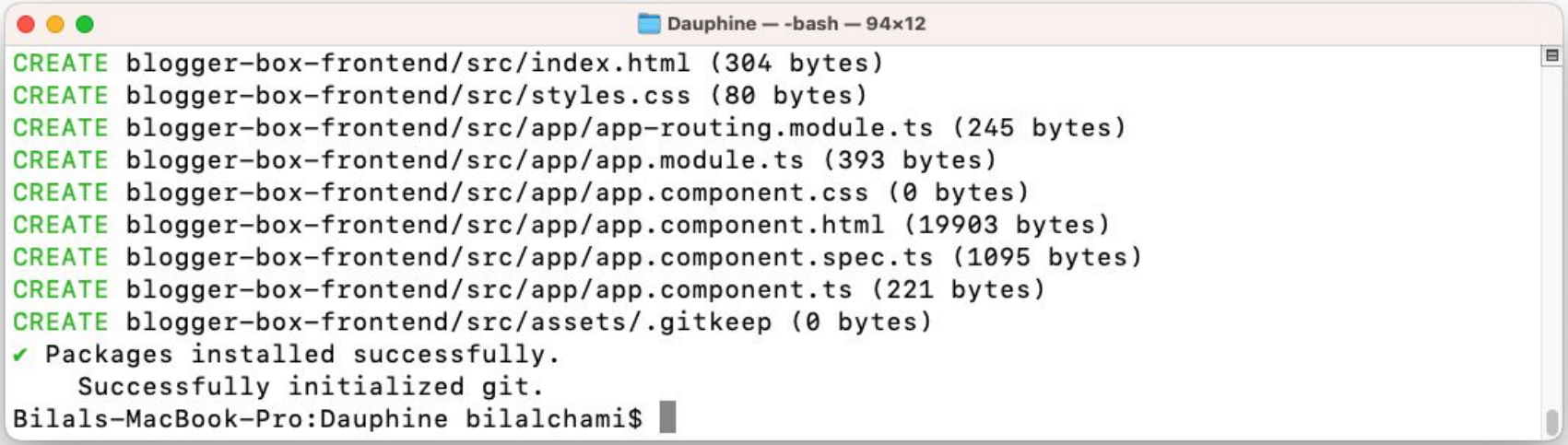
- A new workspace, with a root directory named **blogger-box**
- An initial skeleton application project in the **src/app** subdirectory

Stylesheet format



Server-Side Rendering (SSR)

Project Generation



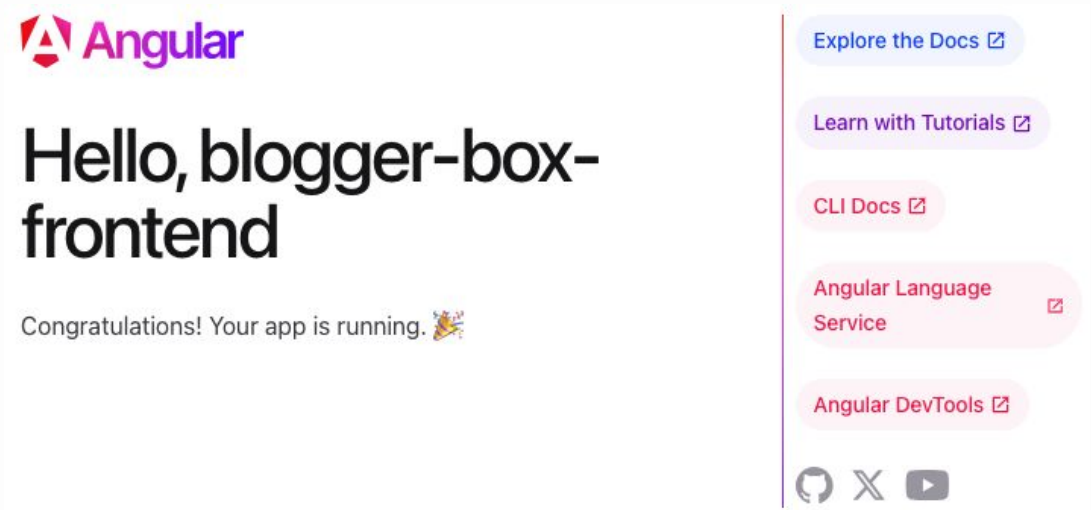
Start the Project

Go to the workspace directory and launch the application:

```
cd blogger-box-frontend
```

```
code .
```

```
ng serve --open
```



Quick Analysis of the Generated Files

main.ts: The startup file of the project

TS main.ts

src > TS main.ts > ...

You, 3 minutes ago | 1 author (You)

```
1  import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';
2  import { AppModule } from './app/app.module';
3
4  platformBrowserDynamic().bootstrapModule(AppModule, {
5    ngZoneEventCoalescing: true,
6  })
7    .catch(err => console.error(err));
```

index.html: The main html file that contains all the elements of the app

```
1   <!doctype html>
2   <html lang="en">
3   <head>
4     <meta charset="utf-8">
5     <title>BloggerBoxFrontend2</title>
6     <base href="/">
7     <meta name="viewport" content="width=device-width, initial-scale=1">
8     <link rel="icon" type="image/x-icon" href="favicon.ico">
9   </head>
10  <body>
11    <app-root></app-root>
12  </body>
13  </html>
```

app.module.ts: The main module of the app

```
1   import { NgModule } from '@angular/core';
2   import { BrowserModule } from '@angular/platform-browser';
3
4   import { AppRoutingModule } from './app-routing.module';
5   import { AppComponent } from './app.component';
6
        You, 9 minutes ago | 1 author (You)
7   @NgModule({
8     declarations: [
9       AppComponent
10    ],
11    imports: [
12      BrowserModule,
13      AppRoutingModule
14    ],
15    providers: [],
16    bootstrap: [AppComponent]
17  })
18  export class AppModule { }
```

app-routing.module.ts: A module containing the routes that will be used in the app

```
1   import { NgModule } from '@angular/core';
2   import { RouterModule, Routes } from '@angular/router';
3
4   const routes: Routes = [];
5
    You, 11 minutes ago | 1 author (You)
6   @NgModule({
7     imports: [RouterModule.forRoot(routes)],
8     exports: [RouterModule]
9   })
10  export class AppRoutingModule { }
```

app.ts/html/css: The Component that contains

```
You, 6 minutes ago | 1 author (You)
1  import { Component, signal } from '@angular/core';
2
   You, 6 minutes ago | 1 author (You)
3  @Component({
4    selector: 'app-root',
5    templateUrl: './app.html',
6    standalone: false,
7    styleUrl: './app.css'
8  })
9  export class App {
10   protected readonly title =
11     signal('blogger-box-frontend');
12 }
```

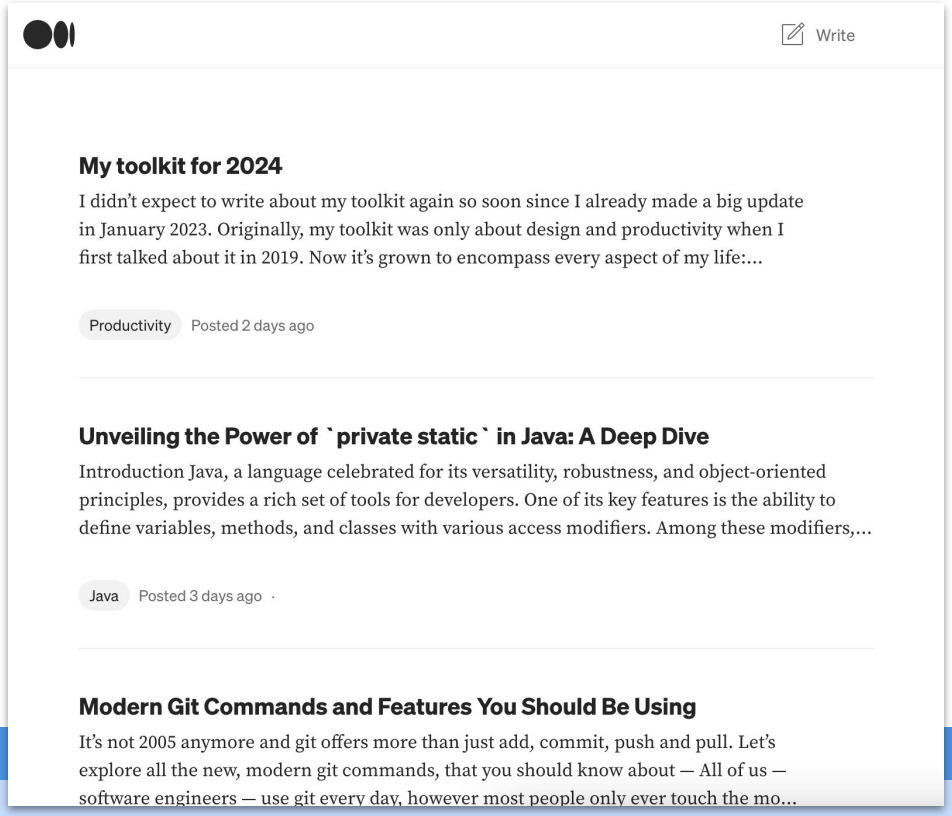
```
<style>…
</style>

<main class="main">
  <div class="content">
    <div class="left-side">
      <svg…
      </svg>
      <h1>Hello, {{ title }}</h1>
      <p>Congratulations! Your app is running. 🎉</p>
    </div>
    <div class="divider" role="separator" aria-label="Divider"></div>
    <div class="right-side">
      <div class="pill-group">
        @for (item of [
          { title: 'Explore the Docs', link: 'https://angular.dev' },
          { title: 'Learn with Tutorials', link: 'https://angular.dev/tutorials' },
          { title: 'CLI Docs', link: 'https://angular.dev/tools/cli' },
          { title: 'Angular Language Service', link: 'https://angular.dev/tools/language-service' },
          { title: 'Angular DevTools', link: 'https://angular.dev/tools/devtools' },
        ]; track item.title) {
          <a
            class="pill"
            [href]="item.link"
            target="_blank"
            rel="noopener"
          >
```

07

Start the Blogging Platform App

Components and Templates



Given the following blogging platform, how many sections/components do you think we can split it into?

Components and Templates

The blogging platform can be split into 3 components.

1. Top Bar
2. Posts List
3. Post List Item

Remove Boilerplate Content

Remove all the content from the main App Component:

- app.css
- app.spec.ts
- app.html
    - Make sure to keep the `<router-outlet />` tag which is a placeholder that lets you load the components dynamically based on the current route state.

Add the main sections/components

As mentioned before, the blogging platform can be split into 3 components.

1. Top Bar
2. Posts List
3. Post List Item

Top Bar Component

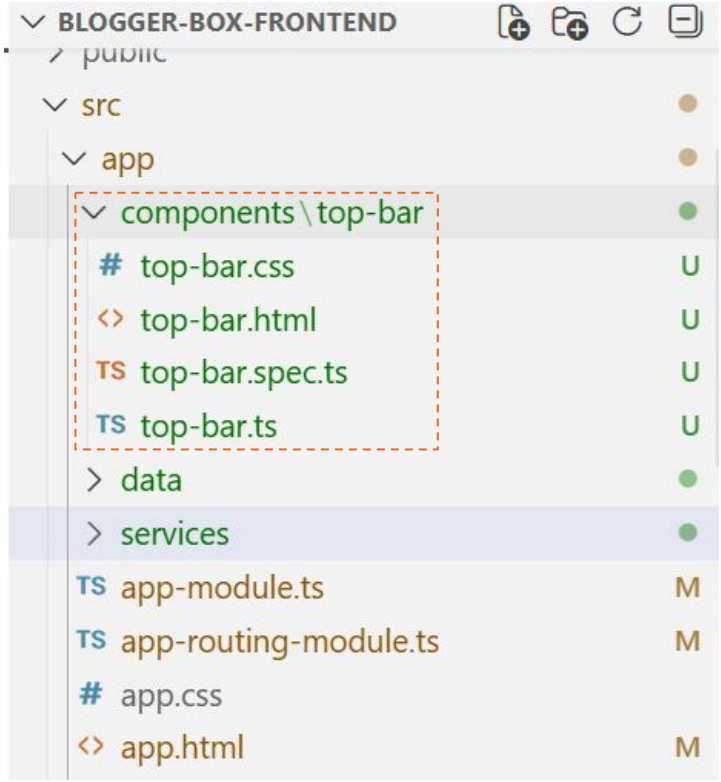
To create the **Top Bar Component**, we will execute the following command:

```
ng generate c components/top-bar
```

This will add the four files to a folder named after the component, which will be located within another folder called "components."

Project structures can vary from one project to another, and the developer has the freedom to choose the structure that best suits their needs.

In our example, all components will be declared within the **`app.module.ts`** file.



Top Bar Component

top-bar.html U

src > app > components > top-bar > <>

Go to component

```
1  <a [routerLink]="['/']">
2      <h1>Blogger Box</h1>
3  </a>
4
5  <button>Write</button>
```

top-bar.ts U

src > app > components > top-bar > TS top-bar.ts

```
2
3   @Component({
4     selector: 'app-top-bar',
5     standalone: false,
6     templateUrl: './top-bar.html',
7     styleUrl: './top-bar.css',
8   })
9   export class TopBar {
10
11  }
```

Top Bar Component

In order to make the component visible by the compiler, it is added to the **`app.module.ts`**.

Adding the **`AppRoutingModule`** indicates that this app will load components dynamically as defined in our custom routes module using Angular Router.

And every newly created component should be added to the module declaration.

```
import { NgModule,
  provideBrowserGlobalErrorListeners } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

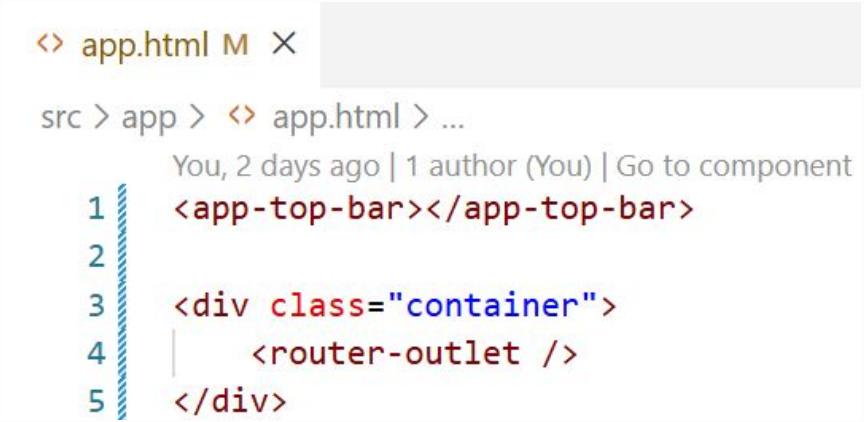
import { AppRoutingModule } from './app-routing-module';
import { App } from './app';
import { TopBar } from './components/top-bar/top-bar';

...
@NgModule({
  declarations: [
    App,
    TopBar,
  ],
  imports: [
    BrowserModule,
    AppRoutingModule
  ],
  providers: [
    provideBrowserGlobalErrorListeners(),
  ],
  bootstrap: [App]
})
export class AppModule { }
```

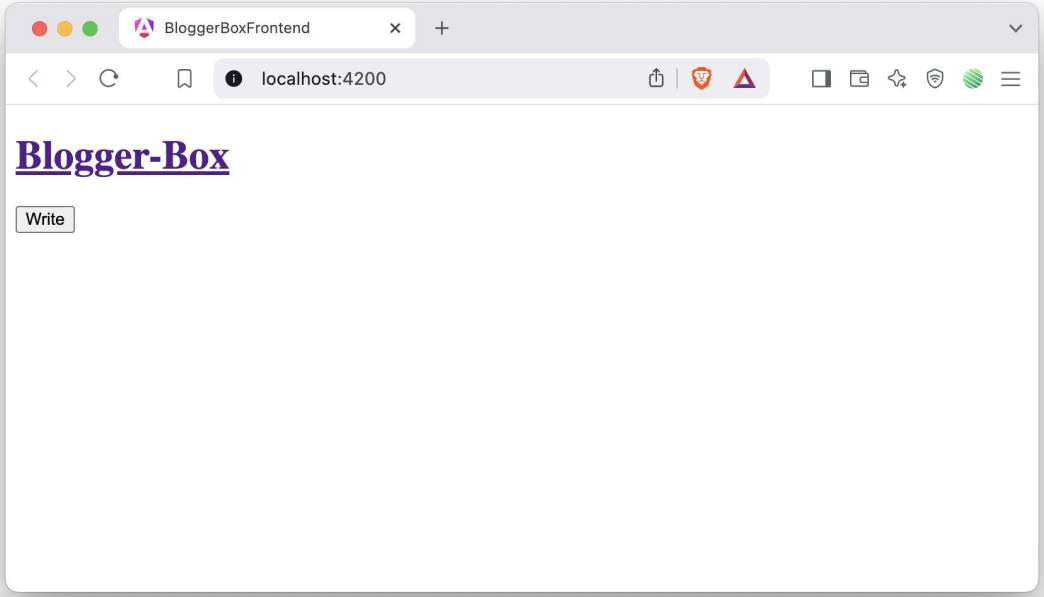
Top Bar Component

The last step will be to add the `app-top-bar` tag to the main **`app.html`**.

Notice how the two main sections were separated.
The **top bar component** will be static and won't be affected by the router.
And every other dynamic component will be loaded in the `<div class="container">` component where the `<router-outlet />` is placed.



Top Bar Component



Styling - Bootstrap

Bootstrap is a popular CSS framework that simplifies and accelerates web development. It offers pre-designed templates, reusable components, and responsive layouts, allowing developers to create sleek and responsive websites with ease.

Add **Bootstrap** and **Bootstrap-icons** to the project.

```
$ npm install bootstrap bootstrap-icons
```.

Styling - Bootstrap

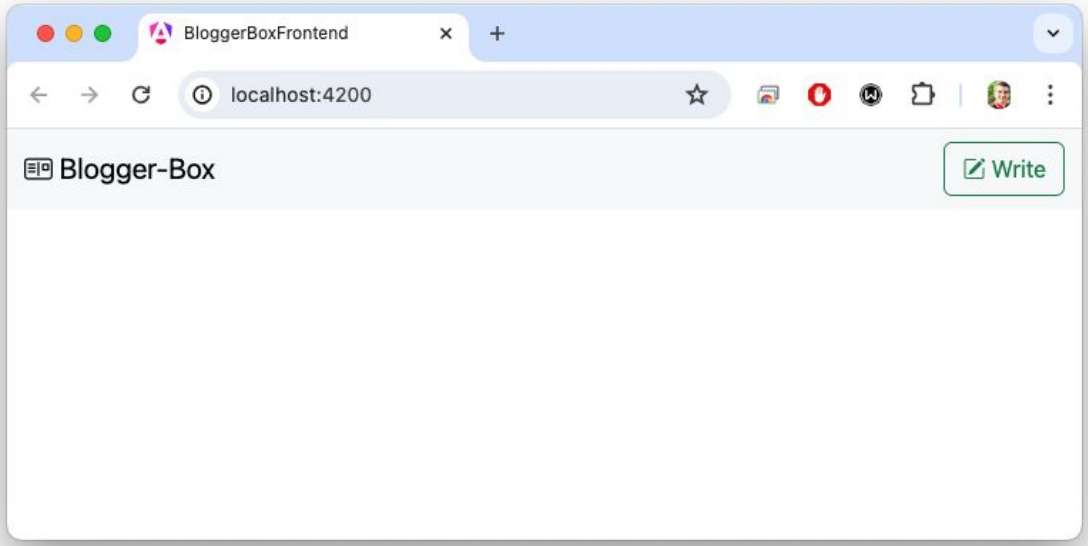
After install, we will configure the **Bootstrap** and **Bootstrap-icons** libraries. Change the **angular.json** file at the build options level and add the **bootstrap.scss**, **bootstrap-icons.css** and **bootstrap.bundle.min.js** files as below:

```
"styles": [
    "node_modules/bootstrap/scss/bootstrap.scss",
    "node_modules/bootstrap-icons/font/bootstrap-icons.css",
    "src/styles.css"
],
"scripts": [
    "node_modules/bootstrap/dist/js/bootstrap.bundle.min.js"
]
```

Add styling the Top Bar Component

```
<nav class="navbar bg-body-tertiary">
 <div class="container-fluid">
   <a class="navbar-brand" [routerLink]="['/']">
     <i class="bi bi-postcard"></i> Blogger-Box
   </a>
   <form class="d-flex">
     <button class="btn btn-outline-success" type="button">
       <i class="bi bi-pencil-square"></i> Write
     </button>
   </form>
 </div>
</nav>
```

Add styling the Top Bar Component



Quick Glimpse at Types in TypeScript

TypeScript offers all of JavaScript's features with additional typing layer.

Types by Inference:

```
let helloWorld = "Hello World";
```

```
let helloWorld: string
```

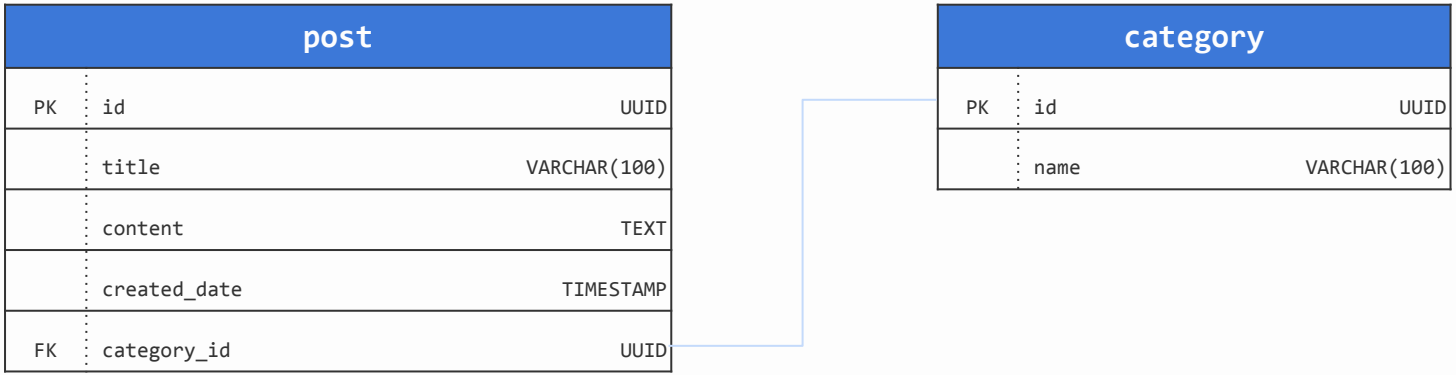
Defining Types, Given the following JSON object:

```
const user = {
  name: "Hayes",
  id: 0,
};
```

We can describe this object's shape using an interface declaration:

```
interface User {
  name: string;
  id: number;
}
```

Data Transfer Objects - DTOs



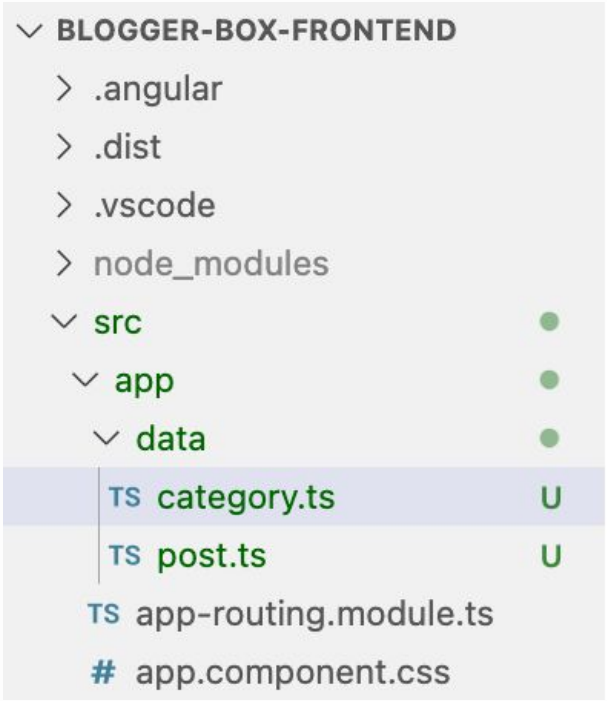
Data Transfer Objects - DTOs

DTOs are used to **encapsulate** and **transfer data** between the **backend and the frontend layers**.

Every defined **Entity** on the backend that will be used in the frontend should have its equivalent DTO.

In our case, we will add a **`data`** folder under the **`app`** directory.

This folder will store all the interfaces used as DTOs.



Data Transfer Objects - DTOs

TS category.ts U

src > app > data > TS category.ts > ...

```ts
export interface Category {
  id: string;
  name: string;
}
```

TS post.ts U

src > app > data > TS post.ts > ...

```ts
import { Category } from "./category";

export interface Post {
  id: string;
  title: string;
  content: string;
  createdDate: Date;
  category: Category;
}
```

Get data from a server

Even if the Backend is not ready, we will enable the HTTP services for our Angular app.
`HttpClient` is Angular's mechanism that let us communicate with a remote server **(API)** over HTTP.

Make `HttpClient` available everywhere in the application by adding the providing function `provideHttpClient()` in the list of providers in the `AppModule`.

Injectables - Post Service

Angular **injectables** are **services** or **objects** that are managed by Angular's **dependency injection** system. They allow for the modularization and reuse of code by providing a way to inject dependencies into components, services, and other Angular constructs. This promotes code maintainability, scalability, and testability in Angular applications.

```
ng generate service services/post.service
```

```
1  import { Injectable } from '@angular/core';
2
3  @Injectable({
4    providedIn: 'root',
5  })
6  export class PostService {
7
8  }
```

Post Service

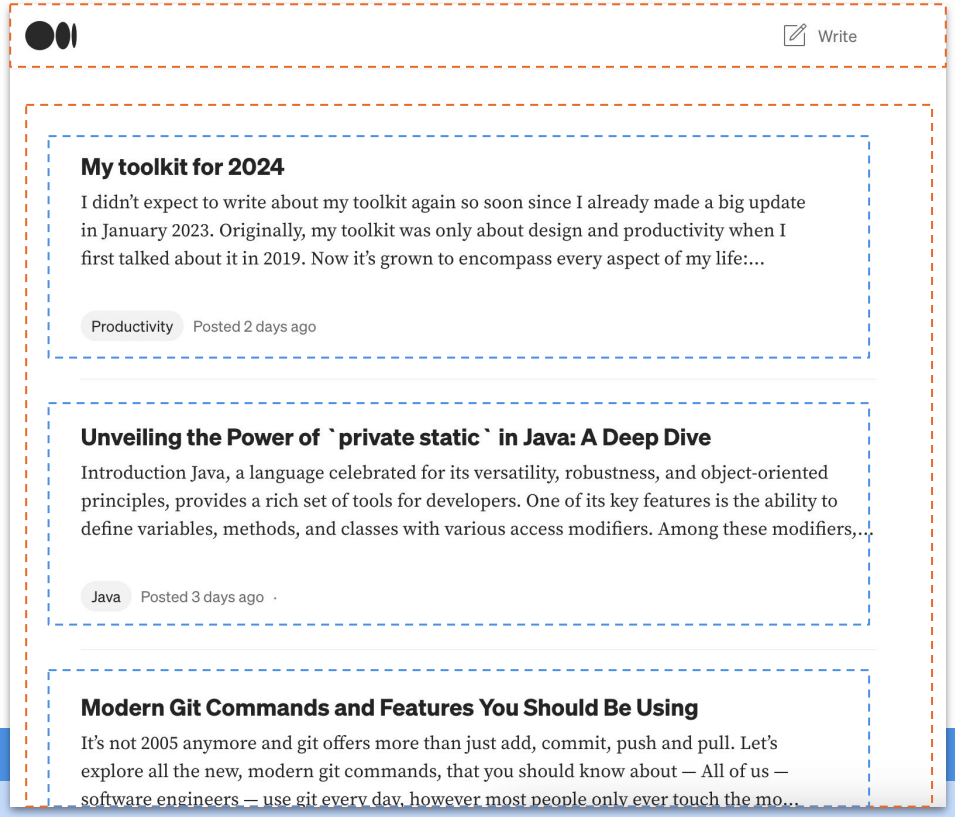
In order to make our service class functional when the backend is not ready, the functions can return mocked data.

You can find the **post.ts** and **category.ts** files with the mocked data on the following gist.

https://gist.github.com/bilal-elchami/2e88bab6b5b9bc5922aa26733d549079

```
1  import { HttpClient } from '@angular/common/http';
2  import { Injectable } from '@angular/core';
3  import { Observable, of } from 'rxjs';
4  import { Post, POSTS } from '../data/post';
5
6  @Injectable({
7    providedIn: 'root',
8  })
9  export class PostService {
10   constructor(private http: HttpClient) {}
11
12   getPosts(): Observable<Post[]> {
13     return of(POSTS);
14   }
15 }
```

List all Posts on the Home Page



After incorporating the "Top Bar" component, our priority should shift to the **main section**, which will house all the posts.

To accomplish this, we need to develop the **PostList** component and integrate it into the home page.

List all Posts on the Home Page

Create a new post-list component by running the command:

```
ng generate c components/post-list
```

This will generate the following files:

- **post-list.css**
- **post-list.html**
- **post-list.spec.ts**
- **post-list.ts**

src > app > components > post-list > TS post-list.ts > ...

```ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-post-list',
  standalone: false,
  templateUrl: './post-list.html',
  styleUrl: './post-list.css',
})
export class PostList {

}
```

List all Posts on the Home Page

Once the component is created, we need to instruct the app to load it on the home page. This can be achieved by including the component in the desired route within the **AppRoutingModule**.

```
1  import { NgModule } from '@angular/core';
2  import { RouterModule, Routes } from '@angular/router';
3  import { PostList } from './components/post-list/post-list';
4
5  const routes: Routes = [
6    { path: '', component: PostList },
7  ];
8
   You, 42 minutes ago | 1 author (You)
9  @NgModule({
10   imports: [RouterModule.forRoot(routes)],
11   exports: [RouterModule]
12 })
13 export class AppRoutingModule { }
```

We should add the new component to the components declarations.

```
@NgModule({
  declarations: [
    App,
    TopBar,
    PostList,
  ],
```

List Posts

We will declare a property on the class level that will store all the posts observable which will be fetched using the `PostService`.

The class implements the `OnInit` interface which is a lifecycle hook that is called after Angular has initialized all properties of a the component.

```
import { Component, OnInit } from '@angular/core';
import { Post } from '../../data/post';
import { PostService } from '../../services/post.service';
import { Observable } from 'rxjs';

You, 1 hour ago | 1 author (You)
@Component({
  selector: 'app-post-list',
  standalone: false,
  templateUrl: './post-list.html',
  styleUrl: './post-list.css',
})
export class PostList implements OnInit {

  posts$: Observable<Post[]> = new Observable();

  constructor(private postService: PostService) {}

  ngOnInit(): void {
    this.posts$ = this.postService.getPosts();
  }
}
```

List Posts

In the **post-list.html** file, we will subscribe to the posts Observable, which is simply a way to start and execute it and deliver its values asynchronously.

We can then verify whether the list of posts contains any elements. If it does, we can loop through them to display each post's content separately.

We'll also add a section indicating that no posts are available to display.

```
@if (posts$ | async; as posts) {
    @if (posts.length) {
        <h2>Blog Posts</h2>
        <ul>
            @for (post of posts; track $index) {
                <li>
                    <h3>{{ post.title }}</h3>
                    <p>{{ post.content }}</p>
                    <p>{{ post.createdDate }}</p>
                    <p>{{ post.category.name }}</p>
                </li>
            }
        </ul>
    }
} @else {
    <p>No posts available.</p>
}
```

Angular directives

Directives are classes that add additional behavior to elements in your Angular applications. The different types of Angular directives are:

DIRECTIVE TYPES	DETAILS
Components	Used with a template. This type of directive is the most common directive type.
Attribute directives	Change the appearance or behavior of an element, component, or another directive.
Structural directives	Change the DOM layout by adding and removing DOM elements.

Built-in attribute directives

Attribute directives listen to and modify the behavior of other HTML elements, attributes, properties, and components.
Many attribute directives are defined through modules such as the `CommonModule`, `RouterModule` and `FormsModule`.

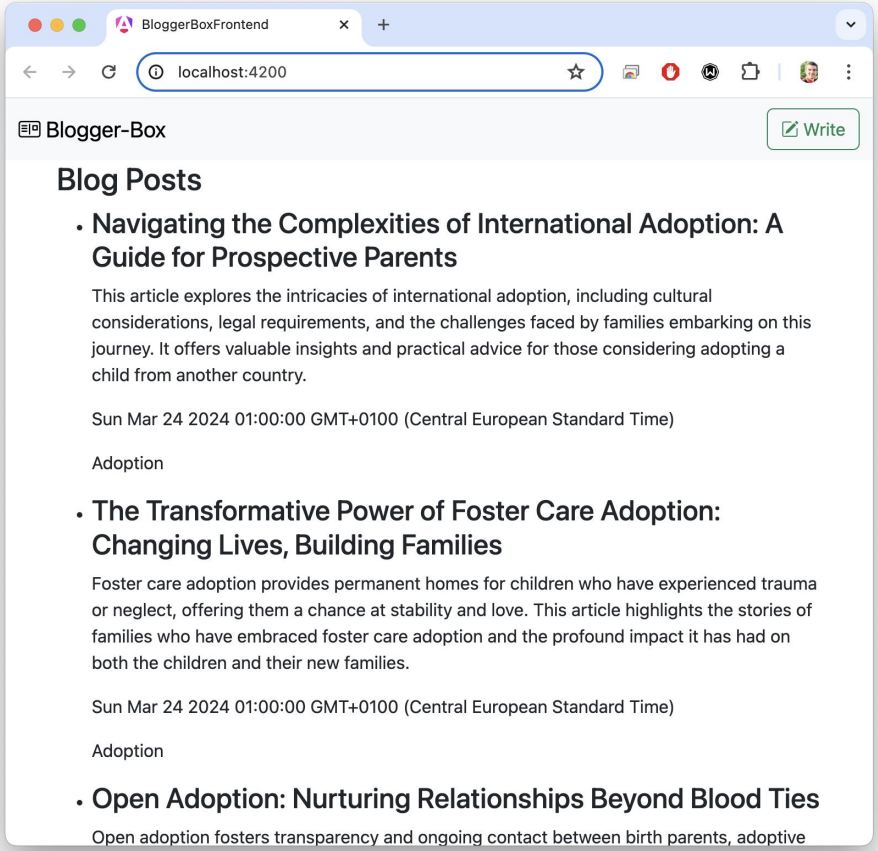
ngClass from `CommonModule` example:

```
<!-- toggle the "special" class on/off with a property -->
<div [ngClass]="isSpecial ? 'special' : ''">This div is special</div>
```

Built-in attribute directives

Displaying and updating properties with **ngModel** from `FormsModule`.

```
<label for="example-ngModel">[(ngModel)]:</label>
<input [(ngModel)]="currentItem.name" id="example-ngModel">
```


BloggerBoxFrontend
localhost:4200
Blogger-Box
Write
Blog Posts
Navigating the Complexities of International Adoption: A Guide for Prospective Parents
This article explores the intricacies of international adoption, including cultural considerations, legal requirements, and the challenges faced by families embarking on this journey. It offers valuable insights and practical advice for those considering adopting a child from another country.
Sun Mar 24 2024 01:00:00 GMT+0100 (Central European Standard Time)
Adoption
The Transformative Power of Foster Care Adoption: Changing Lives, Building Families
Foster care adoption provides permanent homes for children who have experienced trauma or neglect, offering them a chance at stability and love. This article highlights the stories of families who have embraced foster care adoption and the profound impact it has had on both the children and their new families.
Sun Mar 24 2024 01:00:00 GMT+0100 (Central European Standard Time)
Adoption
Open Adoption: Nurturing Relationships Beyond Blood Ties
Open adoption fosters transparency and ongoing contact between birth parents, adoptive

Post List Item

We can take it a step further by creating a separate component for each individual blog post.

This approach enhances code organization, encapsulating post-specific functionalities in one dedicated place and streamlining the List Component.

```
ng generate c components/post-list-item
```

TS post-list-item.ts U ×

src > app > components > post-list-item > TS post-list-item.ts > ...

```ts
import { Component, Input } from '@angular/core';
import { Post } from '../../data/post';

@Component({
  selector: 'app-post-list-item',
  standalone: false,
  templateUrl: './post-list-item.html',
  styleUrl: './post-list-item.css',
})
export class PostListItem {
  @Input() post!: Post;
}
```

Post List Item

`@Input()` is a decorator used to pass data from a parent component to a child component. It enables communication between components by allowing the parent to bind data to properties of the child component.

When an attribute is preceded with `@Input()`, it becomes available on the HTML tag of the component.

TS post-list-item.ts U

src > app > components > post-list-item > TS post-list-item.ts > ...

```ts
import { Component, Input } from '@angular/core';
import { Post } from '../../data/post';

@Component({
  selector: 'app-post-list-item',
  standalone: false,
  templateUrl: './post-list-item.html',
  styleUrl: './post-list-item.css',
})
export class PostListItem {
  @Input() post!: Post;
}
```

```html
<app-post-list-item [post]="post">
</app-post-list-item>
```

Post List Item

The newly created component needs to be declared in the **`NgModule`**, just like the other components.

Once declared, we can utilize it in the **`PostList`** to streamline the code.

Additionally, notice utilizing the **date pipe** to format the post date. Angular Pipes offer convenient tools for string transformation.

For further information, consult the documentation.

post-list-item.html U

src > app > components > post-list-item > post-list-item.html > div.pt-4.p-2 > div > span

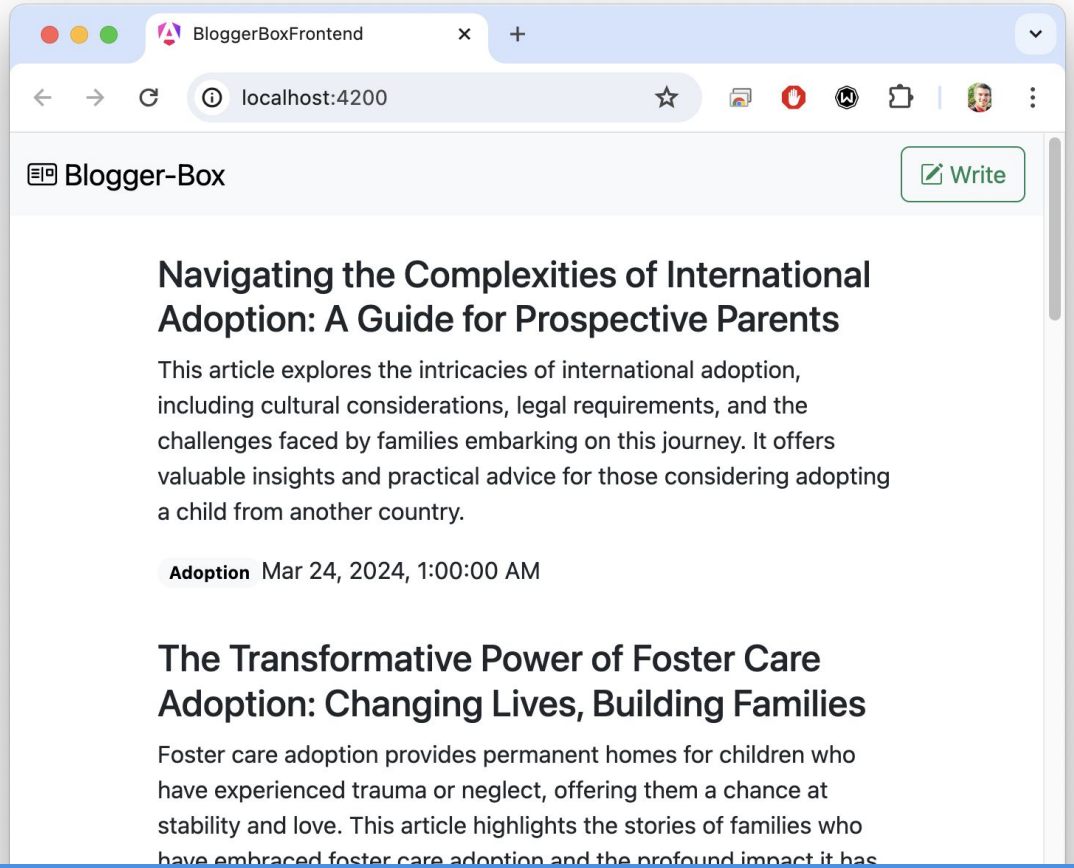
```
@if (post) {
    <div class="pt-4 p-2">
        <h3>{{ post.title }}</h3>
        <p>{{ post.content }}</p>
        <div>
            <span class="badge rounded-pill text-bg-light">
                {{ post.category.name }}
            </span>
            <span>{{ post.createdDate | date : "medium"}}</span>
        </div>
    </div>
}
```

post-list.html M

src > app > components > post-list > post-list.html > ...

You, 2 hours ago | 1 author (You) | Go to component

```
@if (posts$ | async; as posts) {
    @if (posts.length) {
        @for (post of posts; track $index) {
            <app-post-list-item [post]="post"></app-post-list-item>
        }
    }
} @else {
    <p>No posts available.</p>
}
```


BloggerBoxFrontend
localhost:4200
Blogger-Box
Write
Navigating the Complexities of International Adoption: A Guide for Prospective Parents
This article explores the intricacies of international adoption, including cultural considerations, legal requirements, and the challenges faced by families embarking on this journey. It offers valuable insights and practical advice for those considering adopting a child from another country.
Adoption Mar 24, 2024, 1:00:00 AM
The Transformative Power of Foster Care Adoption: Changing Lives, Building Families
Foster care adoption provides permanent homes for children who have experienced trauma or neglect, offering them a chance at stability and love. This article highlights the stories of families who